

PROYECTO RA 2

Índice:

1. Documento de especificación de requisitos	2
Introducción y Propósito	2
1.1 Alcance del Sistema	2
1.2 Usuarios Identificados	2
Requisitos Funcionales (RF)	2
2.1 Módulo de Usuarios y Perfiles	2
2.2 Módulo de Gestión de Inmuebles	2
2.3 Módulo de Búsqueda y Reserva	3
2.4 Módulo de Pagos y Seguridad Financiera	3
Requisitos No Funcionales (RNF)	3
3.1 Seguridad y Privacidad	3
3.2 Rendimiento y Disponibilidad	3
3.3 Usabilidad y Compatibilidad	4
Restricciones del Sistema	4
2. Estudio de viabilidad técnica y económica	4
Viabilidad Técnica	4
Viabilidad Económica	4
3. Diseño de arquitectura del sistema	5
4. Modelos de datos	5
Modelo Entidad-Relación (ER)	5
Modelo Relacional	6
Casos de Uso	6
Diagrama de Clases	6
Diagrama de Secuencia (Proceso de Reserva)	6
6. Planificación temporal y fases del proyecto	6
Fase 1: Planificación y Diseño (4-6 Semanas)	7
Fase 2: Desarrollo del Backend y Base de Datos (8-10 Semanas)	7
Fase 3: Desarrollo del Frontend e Integración (8-10 Semanas)	7
Fase 4: Pruebas, Lanzamiento y Puesta en Marcha (6 Semanas)	8
7. Recursos humanos y materiales estimados	8
Recursos Humanos	8
Recursos Materiales	8

1. Documento de especificación de requisitos

Introducción y Propósito

El objetivo de este documento es definir de manera exhaustiva las capacidades y restricciones del sistema **Stugether**. La plataforma busca resolver la fricción en el mercado de alquiler estudiantil mediante un entorno verificado, con contratos adaptados al calendario académico y un sistema de pagos protegidos.

1.1 Alcance del Sistema

El sistema comprende una plataforma web responsive y aplicaciones móviles que gestionan el ciclo de vida completo del alquiler: desde la prospección geolocalizada hasta la finalización del contrato y la devolución de fianzas.

1.2 Usuarios Identificados

- **Estudiante (Inquilino):** Usuario nacional o internacional que busca alojamiento temporal por motivos de estudio.
- **Propietario (Anfitrión):** Particular o gestor inmobiliario que ofrece inmuebles o habitaciones para el segmento universitario.
- **Administrador de Stugether:** Personal interno encargado de la validación de activos, mediación en disputas y mantenimiento técnico.

Requisitos Funcionales (RF)

2.1 Módulo de Usuarios y Perfiles

- **RF-01 Registro y Autenticación:** El sistema debe permitir el registro mediante correo electrónico o proveedores sociales (Google/Apple). Se debe distinguir el rol de usuario desde el inicio.
- **RF-02 Verificación de Identidad (KYC):** Es obligatorio que los propietarios carguen un documento de identidad oficial y título de propiedad o autorización de gestión. Los estudiantes deben cargar su matrícula universitaria o carta de aceptación.
- **RF-03 Gestión de Perfil:** Los usuarios podrán editar sus datos biográficos, fotos de perfil y preferencias de convivencia (en el caso de estudiantes).

2.2 Módulo de Gestión de Inmuebles

- **RF-04 Publicación Multimedia:** Los propietarios deben poder subir fotografías, vídeos en formato 360° y descripciones detalladas.
- **RF-05 Configuración de Calendario Académico:** El sistema permitirá bloquear fechas específicas, priorizando alquileres de septiembre a junio o semestres cerrados.

- **RF-06 Gestión de Inventario:** El propietario detallará el equipamiento específico para el estudio (escritorio, silla ergonómica, tipo de conexión a internet).

2.3 Módulo de Búsqueda y Reserva

- **RF-07 Motor de Búsqueda Geolocalizado:** Los estudiantes podrán buscar por nombre de universidad, visualizando en un mapa los pisos más cercanos y el tiempo de desplazamiento a pie o en transporte público.
- **RF-08 Sistema de Filtros Avanzados:** Incluye filtros por precio, número de compañeros, género (solo chicas/chicos), aceptación de mascotas y gastos incluidos.
- **RF-09 Solicitud y Aceptación de Reserva:** El estudiante envía una propuesta de reserva. El propietario tiene un máximo de 24 horas para aceptar o rechazar antes de que la solicitud expire.

2.4 Módulo de Pagos y Seguridad Financiera

- **RF-10 Pasarela de Pagos (Escrow):** El sistema procesará el pago del primer mes y la fianza. Stugether retendrá estos fondos en una cuenta segura.
- **RF-11 Desembolso Diferido:** El pago solo se transferirá al propietario 48 horas después de la entrada del estudiante, siempre que no haya una reclamación por discrepancia grave con el anuncio.
- **RF-12 Gestión de Mensualidades:** Automatización del cobro de las rentas mensuales mediante tarjeta de crédito o domiciliación bancaria.

Requisitos No Funcionales (RNF)

3.1 Seguridad y Privacidad

- **RNF-01 Cumplimiento Normativo:** El sistema debe cumplir con el Reglamento General de Protección de Datos (RGPD) y la normativa local de arrendamientos urbanos en cuanto a la gestión de datos.
- **RNF-02 Cifrado de Datos:** Toda la comunicación entre el cliente y el servidor debe viajar bajo el protocolo HTTPS (TLS 1.3) y los datos sensibles en base de datos deben estar cifrados (AES-256).

3.2 Rendimiento y Disponibilidad

- **RNF-03 Alta Disponibilidad:** El sistema debe garantizar un uptime del 99.9%, especialmente crítico durante los meses de julio, agosto y septiembre.
- **RNF-04 Tiempo de Carga:** El tiempo de respuesta de la página principal y los resultados de búsqueda no debe exceder los 2.5 segundos bajo condiciones normales de red.

3.3 Usabilidad y Compatibilidad

- **RNF-05 Diseño Mobile-First:** Dado el perfil joven del usuario principal, la interfaz debe estar optimizada para el uso táctil y pantallas pequeñas.
- **RNF-06 Internacionalización:** La plataforma debe soportar multi-idioma (mínimo español e inglés) y conversión de moneda en tiempo real para estudiantes extranjeros.

Restricciones del Sistema

- **Legales:** La plataforma no actúa como arrendador, sino como intermediario tecnológico, lo que limita su responsabilidad legal sobre el estado físico del inmueble.
- **Técnicas:** El sistema dependerá de APIs de terceros (Stripe, Google Maps), por lo que su operatividad está sujeta a la disponibilidad de estos servicios.

2. Estudio de viabilidad técnica y económica

Viabilidad Técnica

El proyecto es técnicamente viable utilizando tecnologías de mercado maduras. La arquitectura se basará en el uso de PostgreSQL, Python, Django y React.js, permitiendo escalabilidad. La integración de servicios externos está garantizada mediante:

- **Stripe Connect** para la gestión de pagos y depósitos en garantía (*escrow*).
- **Google Maps API** para la geolocalización de las viviendas y distancias a universidades.
- **Cloudinary/AWS S3** para el almacenamiento masivo de imágenes y vídeos 360° de alta resolución.
- **Auth0** para la gestión de identidades y autenticación segura.

Viabilidad Económica

- **Costes de Inversión (CAPEX):** El desarrollo inicial (MVP) requiere una inversión en desarrollo de software, diseño UX/UI y configuración de infraestructura en la nube.
- **Costes Operativos (OPEX):** Mantenimiento de servidores, licencias de APIs de mapas y marketing digital para captación de estudiantes.
- **Modelo de Ingresos:** Se basa en una **comisión por transacción** (Service Fee) cobrada al estudiante (aprox. 5-10%) y una comisión de gestión al propietario, similar al modelo Airbnb pero optimizado para estancias de larga duración. La alta recurrencia anual del mercado estudiantil asegura un flujo de caja previsible.

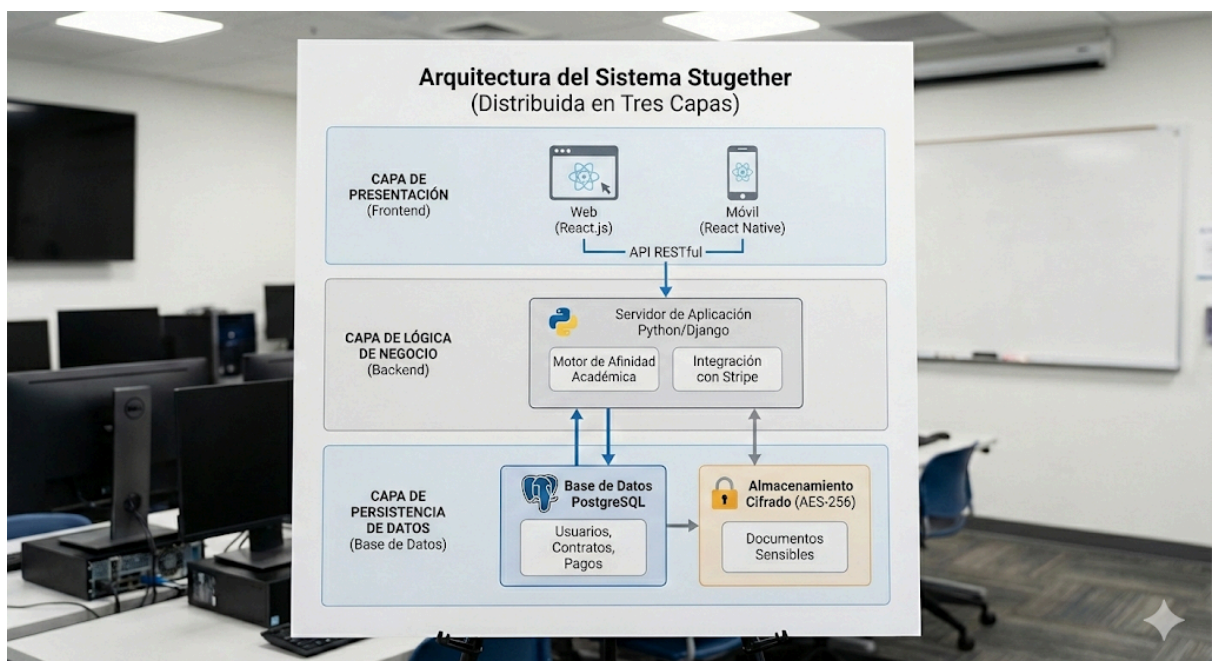
3. Diseño de arquitectura del sistema

El sistema adopta una **arquitectura de tres capas** distribuida en la nube para garantizar independencia y escalabilidad:

Capa de Presentación (Frontend): Desarrollada en **React.js** para la web y **React Native** para móviles. Se comunica con el backend mediante una API RESTful.

Capa de Lógica de Negocio (Backend): Usaremos **Python/Django**, estando al cargo del motor de afinidad académica y la integración con Stripe.

Capa de Datos (Persistencia): Uso de **PostgreSQL** para usuarios, contratos y pagos, con cifrado **AES-256** para documentos sensibles.



4. Modelos de datos

Modelo Entidad-Relación (ER)

El núcleo del sistema se basa en las siguientes entidades principales y sus relaciones:

- **Usuario:** Contiene credenciales, perfil y tipo de rol (Estudiante/Propietario). Un usuario puede tener múltiples Propiedades (si es propietario) o Reservas (si es estudiante).
- **Propiedad:** Almacena dirección, características, precio y estado de verificación. Está vinculada a una Universidad específica por cercanía.

- **Reserva:** Relaciona a un Estudiante con una Propiedad, incluyendo fechas de inicio/fin, estado del pago y contrato firmado.
- **Mensaje:** Gestiona la comunicación interna entre las partes antes y durante la reserva.

Modelo Relacional

- **Tabla Usuarios:** id_usuario (PK), nombre, email, password_hash, rol, documento_identidad.
- **Tabla Propiedades:** id_propiedad (PK), id_propietario (FK), titulo, descripcion, coordenadas, precio_mes, fianza.
- **Tabla Reservas:** id_reserva (PK), id_estudiante (FK), id_propiedad (FK), fecha_inicio, fecha_fin, estado_pago.
- **Tabla Reseñas:** id_reseña (PK), id_reserva (FK), puntuacion, comentario.

5. Diagramas UML

Casos de Uso

El diagrama define las interacciones principales:

- **Estudiante:** "Buscar piso por universidad", "Solicitar reserva", "Realizar pago", "Valorar estancia".
- **Propietario:** "Publicar anuncio", "Gestionar solicitudes de reserva", "Verificar identidad".
- **Administrador:** "Validar documentación", "Gestionar disputas de pagos".

Diagrama de Clases

Define la estructura interna del software con clases como **PropertyManager** (gestión de lógica de anuncios), **PaymentGateway** (interfaz con Stripe) e **IdentityVerifier** (lógica de validación de documentos).

Diagrama de Secuencia (Proceso de Reserva)

Descripción de flujo: El Estudiante solicita reserva -> El Sistema bloquea fechas -> El Propietario acepta -> El Estudiante paga el primer mes -> El Sistema retiene fondos -> El Propietario recibe fondos tras el Check-in exitoso.

6. Planificación temporal y fases del proyecto

El desarrollo del Producto Mínimo Viable (MVP) se estima en un periodo de **6 a 7 meses**, estructurado en cuatro fases secuenciales y superpuestas, destacar que la implementación y la puesta en marcha de cada fase del proyecto la haremos en conjunto y no distinguiremos en dividir el trabajo, sino que nos coordinaremos para exprimir nuestro potencial y hacer esta odisea mucho más amena.

Fase 1: Planificación y Diseño (4-6 Semanas)

Esta fase inicial se enfoca en la definición precisa del proyecto antes de comenzar la codificación.

- **Objetivos Clave:**
 - Cerrar y validar los requisitos funcionales y no funcionales (RF/RNF).
 - Definir la arquitectura tecnológica completa (Stack).
 - Diseñar la estructura de la base de datos (PostgreSQL).
 - Crear los *wireframes* (bocetos) y *mockups* (diseño visual) de la interfaz de usuario.
- **Hito Clave:** Documento de diseño de arquitectura y *mockups* de UI/UX aprobados.

Fase 2: Desarrollo del Backend y Base de Datos (8-10 Semanas)

Se centra en la construcción del núcleo lógico, la seguridad y la gestión de datos utilizando Python/Django.

- **Objetivos Clave:**
 - Desarrollar la API RESTful para la comunicación entre el *frontend* y el *backend*.
 - Implementar los módulos de Registro y Autenticación (RF1).
 - Construir el módulo de Verificación de Estudiante y documentos (RF2).
 - Integrar la lógica de Publicación y Gestión de Anuncios (RF3).
 - Integrar y probar la pasarela de pago para el cobro de la comisión de servicio (RF8).
- **Hito Clave:** API funcional y Módulo de Verificación de Estudiantes operativos.

Fase 3: Desarrollo del Frontend e Integración (8-10 Semanas)

Esta fase se solapa con la Fase 2 y se enfoca en la interfaz de usuario, la lógica del cliente (React) y la conexión final con el *backend*.

- **Objetivos Clave:**
 - Desarrollar todos los componentes de la interfaz de usuario utilizando React.
 - Implementar la lógica del Sistema de Búsqueda y Filtros Avanzados (RF4), incluyendo la afinidad académica.
 - Construir los módulos de Visualización de Afinidad (RF5) y la Mensajería Interna (RF6).
 - Finalizar el Panel de Control del Usuario y la Gestión de Solicitudes/Reservas (RF7, RF9).
 - Asegurar la total responsividad de la web (RNF6).
- **Hito Clave:** Interfaz de usuario completa e integrada con el *backend*. Funcionalidad principal de búsqueda y reserva operativa.

Fase 4: Pruebas, Lanzamiento y Puesta en Marcha (6 Semanas)

Fase final dedicada a la estabilización, la calidad y el despliegue del producto.

- **Objetivos Clave:**
 - Realizar pruebas exhaustivas de Control de Calidad (QA), Pruebas de Rendimiento (RNF2) y Pruebas de Seguridad (RNF4).
 - Ejecutar una fase de **Beta Cerrada** con un grupo limitado de anfitriones y estudiantes para recoger *feedback* real.
 - Corregir los errores y pulir la experiencia de usuario.
 - Preparar la infraestructura de producción para el despliegue a gran escala.
 - Cargar el contenido inicial de anuncios.
- **Hito Clave:** Lanzamiento oficial del MVP de **STUGETHER** en la ciudad piloto (Mes 7).

7. Recursos humanos y materiales estimados

Recursos Humanos

- **1 Project Manager:** Coordinación general y metodología Agile.
- **2 Desarrolladores Full-stack:** Encargados del frontend y backend.
- **1 Diseñador UX/UI:** Responsable de la experiencia de usuario y diseño visual.
- **1 Especialista en QA/Seguridad:** Pruebas de penetración y validación de flujos financieros.

Recursos Materiales

- **Infraestructura Cloud:** Instancias de servidor (AWS/GCP), bases de datos gestionadas y almacenamiento S3.
- **Herramientas de Desarrollo:** Licencias de IDEs (VS Code), herramientas de diseño (Figma), y gestión de proyectos (Trello).
- **Equipamiento:** Equipos de alto rendimiento para el equipo de desarrollo.